INTERN PROJECT BRIEF

Recruiting & Applicant Tracking System (ATS)

ASP.NET Core web application | Requisitions, Candidates, Pipeline, Interviews

Item	Detail
Project type	Team-based internship project, one ASP.NET Core web application (server-side, no separate API)
Primary technology	C# / ASP.NET Core (MVC or Razor Pages), Entity Framework Core, SQL Server (or PostgreSQL / SQLite for local development)
Suggested team size	3–5 interns, 1 mentor / reviewer
Mentor	_____
Start date / Demo date	_____ / _____

1. Project Overview

Interns will design and build a simplified Applicant Tracking System (ATS) that manages the hiring process from opening a job requisition through to interviewing candidates. The project is a reduced-scope version of real-world recruiting functionality, so interns practise on a realistic business domain with genuine rules around access control, confidential data and workflow.

The whole system is a single ASP.NET Core web application. Pages, business logic and data access all live in the same solution; there is no separate REST API and no separate front-end project. The team is free to choose how the user interface looks and which CSS or JavaScript libraries it uses, as long as it is served by the ASP.NET Core application.

1.1 Goals

- Deliver a working, tested ASP.NET Core web application covering the core hiring workflow.
- Apply a clean layered structure with dependency injection and a service layer for business rules.
- Implement role-based access control and requisition-scoped data visibility.
- Protect confidential data (compensation) and keep an audit trail.
- Work as a team using Git, pull requests and code review.

1.2 Learning Outcomes

- ASP.NET Core MVC or Razor Pages: controllers/handlers, view models, validation, layouts and partial views.
- EF Core: code-first modelling, migrations, relationships, queries and filtering.
- Authentication and authorization using ASP.NET Core Identity and policy-based permissions.
- Business-rule modelling: status workflows, scoping rules and duplicate detection.
- Unit testing business rules with xUnit.
- Documenting and demoing technical work.

2. Core Business Concepts

These rules are the foundation of the system. Everything else depends on them, so the whole team must understand them before writing code.

Concept	What it means
Candidate vs Application	A Candidate is a person in the global talent pool. An Application links that person to exactly one requisition and holds the stage, match score, interviews and offers.
Requisition scope	Leadership (permission REQUISITIONS.ALL_REQS.VIEW) sees all requisitions. Everyone else sees only requisitions where they are on the requisition team (Owner, Recruiter, Technical/Final Interviewer, etc.). Applications and interviews inherit this scope.
Talent pool exception	Pool search can find people across requisitions, but application detail (stages, feedback, compensation) stays scoped.
Compensation confidentiality	Budget and salary expectations are hidden unless the user has ATS.COMPENSATION.VIEW or .EDIT. Every view of compensation is audited.
Pipeline stages	Shared stage catalog: Applied, then working stages (e.g. Screening, Interview), then terminal stages (Hired, Disqualified, NFA). Stage moves are always human-driven.
Source attribution	Every candidate and application records its source (e.g. referral, careers page, agency).
Automation is advisory only	Match scores only assist. The system must never auto-hire or auto-reject.

3. Project Scope

The scope is deliberately focused. The team should finish every Must item with good quality before touching anything else.

3.1 In scope (core deliverables)

- Security foundation: login, roles, permissions, requisition scoping, compensation protection, audit log.
- Requisitions: create, approve, publish, team management, hold/close/fill.
- Candidates and Talent Pool: create with résumé upload, duplicate detection, search.
- Applications and Pipeline: add a candidate to a requisition, move stages, disposition with reason, scoped pipeline board.
- Interviews: schedule, assign panel, submit feedback with independence rule, My Interviews page.

3.2 Stretch goals (only if every Must item is done)

- Merge duplicate candidates.
- Simple match score between requisition skills and candidate skills (advisory).
- Requisition clone and statistics page.
- Candidate tags and pipeline filters.
- Offers and onboarding hand-off.

3.3 Out of scope

- Real AI / résumé parsing (use manual entry).
- Calendar synchronisation and real email delivery (log or simulate).
- Automatic interviewer slot discovery.
- Job board auto-posting.
- Multi-level approval engine (use a single approver).
- Recruiting analytics and reporting.
- A separate REST API or separate front-end application.

4. Technical Requirements

Area	Requirement
Language / runtime	C# on the current .NET LTS version
Application type	Single ASP.NET Core web application using MVC or Razor Pages. No REST API layer; pages call services directly.
User interface	Team's choice of styling and client-side libraries, delivered from within the ASP.NET Core application. Must be usable and consistent across pages.
Data	Entity Framework Core code-first with migrations; SQL Server or PostgreSQL (SQLite acceptable for local development)
Architecture	Separate presentation, services (business rules) and data access. Use dependency injection. No business logic in controllers, page models or views.
Security	ASP.NET Core Identity with cookie authentication; permission-based authorization policies; anti-forgery tokens on all forms
Validation	Server-side validation with data annotations or FluentValidation, with friendly error messages on forms

Area	Requirement
Testing	xUnit unit tests for the main business rules (scope, compensation, stage moves, feedback independence)
File handling	Résumé upload (PDF only, size limit) saved to local storage behind an interface so it can be swapped later
Source control	Git with feature branches and pull requests; at least one reviewer per PR
Logging	Built-in ILogger or Serilog

5. Functional Requirements

Priority key: M = Must have, S = Should have, C = Could have. FR-IDs identify each requirement for traceability.

5.1 ATS Foundation and Security

FR ID	Requirement	Priority
FR-ATS-001	Enforce requisition scope: leadership sees all, others see only requisitions where they are on the team. Applications and interviews inherit the scope.	M
FR-ATS-002	Hide compensation fields unless the user has the compensation permission; log each compensation view in an audit table.	M
FR-ATS-003	Maintain a pipeline stage catalog with reserved stages (Applied, Hired, Disqualified, NFA) and admin-managed working stages.	M
FR-ATS-004	Record the source of every candidate and application.	S
FR-ATS-005	Match scores are labelled advisory; no automatic hire or reject.	M

5.2 Requisitions

FR ID	Requirement	Priority
FR-REQ-001	Create a requisition in Draft (title, department, type, seniority, skills, job description, hiring manager). Creator becomes Owner. No budget at creation.	M
FR-REQ-002	Single approver approves or rejects; rejection returns the requisition to Draft with a comment.	M
FR-REQ-003	Publish a requisition. Only published requisitions accept applications.	M
FR-REQ-004	Requisition team (Owner, Recruiters, Technical/Final Interviewers). Exactly one Owner. Team changes are audited. Team drives scope.	M
FR-REQ-007	Budget / salary range is confidential and permission-gated.	M
FR-REQ-009	Hold, close, cancel and fill a requisition, with a reason where required.	M
FR-REQ-008	Edit rules depend on status; material changes after publish require re-approval.	C
FR-REQ-010	Clone a requisition into a new Draft (criteria copied; team and applications not copied).	C

5.3 Candidates and Talent Pool

FR ID	Requirement	Priority
FR-CAND-001	Create a candidate with a PDF résumé upload; match an existing candidate where one already exists.	M
FR-CAND-003	Detect duplicates by email and phone number and warn before saving.	M
FR-CAND-005	Talent pool search across all requisitions for users with pool view permission.	M
FR-CAND-006	Capture candidate consent and store the date.	S
FR-CAND-008	Treat personal data as sensitive: restrict access and avoid writing it to logs.	M
FR-CAND-004	Merge duplicate candidates without losing applications.	C
FR-CAND-012	Tag candidates and filter by tag.	C

5.4 Applications and Pipeline

FR ID	Requirement	Priority
FR-APP-001	An application is a requisition-scoped link between a candidate and a requisition.	M
FR-APP-002	Add a talent-pool candidate to a published requisition.	M
FR-APP-005	Move an application between stages. Stage moves are user-initiated only.	M
FR-APP-006	Disposition (Disqualified / NFA) requires a reason.	M
FR-APP-009	Pipeline board shows only applications within the user's requisition scope.	M
FR-APP-004	Simple match score (e.g. skill overlap) shown as advisory.	C
FR-APP-011	Filter the pipeline by stage, source or recruiter.	C

5.5 Interviews

FR ID	Requirement	Priority
FR-INT-001	Interviews belong to an application and inherit its requisition scope.	M
FR-INT-002	Schedule an interview (date, time, type, mode). Calendar sync is simulated or logged only.	S
FR-INT-005	Panel is chosen from the requisition's interviewer team; at most one lead interviewer.	M
FR-INT-006	Interviewers submit structured feedback (rating, strengths, concerns, recommendation); one active submission per interviewer.	M
FR-INT-007	Feedback independence: an interviewer cannot see others' feedback until they submit their own.	M
FR-INT-010	My Interviews page: panel members see only the interviews assigned to them.	M
FR-INT-009	Mark an interview as completed or no-show.	S

6. Roles and Permissions

Use permission-based authorization policies rather than hard-coding role names in controllers or views. Suggested seed roles:

Role	Typical permissions
Leadership / Admin	REQUISITIONS.ALL_REQS.VIEW, ATS.COMPENSATION.VIEW/EDIT, manage pipeline stages, approve requisitions
Recruiter	Create/edit requisitions they belong to, manage candidates and applications, schedule interviews
Hiring Manager / Owner	Own requisition, manage team, view applications and feedback on their requisitions
Interviewer	INTERVIEWS.FEEDBACK.ADD, view My Interviews, submit feedback for assigned interviews only

7. Suggested Data Model

The team should agree an ERD before coding and have it reviewed by the mentor.

Entity	Key fields / notes
Employee	Id, Name, Email, Department (people who can be on teams or panels); linked to the login user
Requisition	Id, Title, Department, Type, Seniority, JobDescription, Status, Openings, HiringManagerId, Budget (confidential)
RequisitionTeamMember	RequisitionId, EmployeeId, TeamRole (Owner, Recruiter, TechnicalInterviewer, FinalInterviewer)
Candidate	Id, FullName, Email, Phone, ResumePath, ConsentDate, Source
Application	Id, CandidateId, RequisitionId, StageId, Source, DispositionReason, ExpectedSalary (confidential)
PipelineStage	Id, Name, Order, IsTerminal, IsReserved
Interview	Id, ApplicationId, Type, ScheduledAt, Mode, Status
InterviewPanelMember	InterviewId, EmployeeId, IsLead
InterviewFeedback	Id, InterviewId, InterviewerId, Rating, Strengths, Concerns, Recommendation, SubmittedAt
AuditLog	Id, UserId, Action, EntityType, EntityId, Timestamp, Details

8. Suggested Pages / Screens

Screen	Purpose
Login	Sign in with email and password
Requisitions list and detail	Scoped list, create/edit, submit for approval, publish, hold/close/fill, compensation shown only with permission
Requisition team	Add or remove team members and set roles; exactly one Owner
Approvals	Approver sees pending requisitions and approves or rejects with a comment
Talent pool	Search and filter candidates; create candidate with résumé upload and duplicate warning
Candidate detail	Person details, résumé download, applications visible within the user's scope
Pipeline board	Applications grouped by stage for a requisition; move stage; disposition with reason

Screen	Purpose
Schedule interview	Choose date, type and panel from the requisition's interviewers
My Interviews	Interviews assigned to the signed-in interviewer, with feedback form
Admin: stages and users	Manage working pipeline stages, roles and permissions

9. Non-Functional Requirements

- Security: every page and action requires authorization; scope checks happen on the server, never only by hiding buttons.
- Confidentiality: compensation fields are not loaded into the page model or view when the user lacks permission.
- Auditability: significant actions (team changes, stage moves, compensation views) are written to the audit log.
- Usability: consistent layout, clear validation messages, and confirmation for destructive actions.
- Maintainability: consistent naming, small services, no business logic in controllers or views.
- Reliability: validate all inputs and handle errors with a friendly error page.
- Documentation: README with setup steps and an architecture diagram.

10. Suggested Delivery Phases

The team should agree the split of work among members at the start. Each phase should end with something that runs and can be demonstrated.

Phase	Focus	Outcomes
1	Setup and foundation	Repository, solution structure, ERD, seed data, login, roles and permissions, stage catalog, audit log, scope helper
2	Requisitions	Create, approval, publish, team management, status changes, confidential budget
3	Candidates and pipeline	Talent pool, résumé upload, duplicate detection, applications, stage moves, dispositions, scoped pipeline board
4	Interviews and finish	Scheduling, panel, feedback with independence rule, My Interviews, tests, bug fixes, README, final demo

11. Deliverables

1. Source code in a Git repository with a clear commit and PR history.
2. A working web application with seed data so every role can be demonstrated.
3. Database migrations and an ERD diagram.
4. Unit tests for the main business rules.
5. README: setup instructions, architecture overview, and how to run tests.
6. Final demo (about 20 minutes) walking through the hiring flow end to end.
7. Short written reflection from each intern: what they built, what they learned, what they would improve.

12. Evaluation Criteria

Criteria	Weight	What good looks like
Functional completeness	30%	All Must items work end to end; Should items where possible
Correct business rules	25%	Scoping, compensation confidentiality, feedback independence and stage rules behave exactly as specified
Code quality and architecture	20%	Clean layering, readable code, sensible use of patterns and DI
Testing	5%	Meaningful tests for key business rules
Teamwork and process	10%	Regular commits, reviewed PRs, clear task split
Demo and documentation	10%	Clear presentation, accurate README, ability to explain design choices

13. Assumptions and Notes

- Interns already know C# basics and receive a short introduction to ASP.NET Core and EF Core.
- Employees are seeded test data; there is no HR module to build.
- Email, calendar and AI features are simulated so the focus stays on core .NET skills.
- The mentor holds the detailed business requirements and is the reference for any questions on rules and edge cases.
- The mentor may adjust priorities after the initial design review. Quality of Must items matters more than the number of features.